KUET Royale: which algorithm, where, why

CSE 3210 Artificial Intelligence Lab. Eight course algorithms, each with its own job. All of it is plain Python (standard library only) in `ai/kuet_royale/`.

The game in short. Two departments (any of CSE, EEE, ME, CE) fight on a 13×7 grid with a river, two bridges, roads (cost 1), grass (cost 2) and mud (cost 4). Each side has a 4-card deck. Sides take turns playing one card into the top or bottom lane; after each card the units fight for 3 battle steps (up to 15 more after the last card). A leg ends when all 8 cards are played or a tower falls; its score is left tower health minus right tower health. A match is two legs, each side playing first once because playing second is an edge; the side that did more damage to the enemy tower over both legs wins. *Once the decks are dealt, nothing is random, nothing is hidden and the turns alternate*, so the course's searches apply exactly as taught.

Algorithm	Its job in the game	Code	Taught in
CSP backtracking	builds each deck under cost and role limits	`deck_csp.py build_deck`	Lab 06
■ Uniform-cost search	which enemy is cheapest to reach (non-hover units)	`search.py uniform_cost`	course
■ A* search	route of a *careful* unit	`search.py a_star`	Lab 03
■ Greedy best-first	route of a *reckless* unit	`search.py best_first`	Lab 03
■ Breadth-first search	target and route of a *hover* unit; area of a splash	`search.py breadth_first`, `within`	course
Fuzzy inference	which enemy to hit when several are in range	`fuzzy.py TARGET`	Lab 05
Alpha-beta search	brain 1; also the Hint button and move review	`alphabeta.py`	Lab 04 + theory
MCTS (UCB1)	brain 2 (and the Easy level)	`mcts.py`	Lab 08

One turn, start to finish. (1) At the start of the match **CSP** builds both decks. (2) The side to play picks a card and a lane: **alpha-beta**, **MCTS**, or you (Hint asks alpha-beta). (3) Each battle step, every unit does one thing. If an enemy unit or the enemy tower is in range it attacks, and if several targets are in range **fuzzy** rules pick which; a splash also hits every other enemy unit that **breadth-first** finds within one step of the target. Otherwise it picks a target with **uniform-cost** (hover units: **breadth-first**), then walks towards it on the route from **A***, **best-first** or **breadth-first**, according to the card's walking style. Then the towers fire at units where they stood when the step began, and all damage lands at once.

1. CSP backtracking with MRV and LCV

`deck_csp.py` Lab 06

What. Variables: the 4 deck slots *Tank, Ranged, Fast, Free*. Domains: the department's 8 cards that fit the slot (any card fits Free). Constraints: no card twice, and the total cost is 13 or 14. Backtracking assigns a slot, checks the constraints and undoes the choice when nothing fits. *MRV* fills the slot with the fewest legal cards first; *LCV* tries first the card that leaves the other slots the most choices.
Why. Choosing a deck under limits is a textbook CSP. Cost is a hand-set strength rating (nothing in play spends it), so the limit stops a deck taking all the strongest cards. The match seed shuffles the order the cards are read in, so equally good decks vary between matches. Honest note: with these 8-card rosters MRV always ends up with the order Tank, Ranged, Fast, Free, and LCV only saves the odd undo (ME, CE).
Example. CSE as the left side of match seed 4: Compiler, Logic Bomber, Debugger, Stack Runner, found in 4 tries with 0 undone. The intro sheet shows these numbers.

■ 2. Uniform-cost search (UCS)

`search.py uniform_cost, routes.py nearest` course

What. Expands tiles in order of path cost g (a priority queue on g). With many goal tiles (every enemy unit, and the three tiles next to the enemy tower), the first goal taken off the queue is the cheapest one to reach. The game runs it once per start tile and remembers the order, so `routes.py nearest` just takes the goal reached first.
Where. Every battle step, for a non-hover unit with nobody in range: it decides *which* enemy to walk to.
Why. There are several goals and the tiles cost different amounts. Breadth-first would count mud as cheap as road; A* and best-first need a single goal. Shown on screen as blue tiles.

■ 3. A* search

`search.py a_star` Lab 03

What. Expands the tile with the smallest $f = g + h$, where h is the Manhattan distance to the goal. Each step costs at least 1 and changes h by at most 1, so h is consistent (it never overestimates) and the route found is the cheapest.
Where. The route of every *careful* unit (most cards, such as Compiler, Algorithmist and Heavy Machine), once UCS has chosen its target.
Why. We want the cheapest route, and A* finds it while exploring fewer tiles than UCS, because it is aimed at one goal. Shown as yellow tiles.

4. Greedy best-first search

`search.py best_first` Lab 03

What. Expands the tile that *looks* closest, by h alone, and never looks at what the path has cost.
Where. The route of *reckless* units: Stack Runner, Relay Runner, Gear Runner, Mini-Bot, Road Roller and Construction Crew.
Why. It gives units a personality: they head straight at the goal and may charge through mud, and it lets the screen show greedy against optimal. Example (engine coordinates): from the right side's top-lane start $(11, 1)$ to $(1, 3)$ next to the left tower, best-first's route costs **34** where A*'s costs **12**; in play this shows as "Mini-Bot takes the greedy route: it costs 34 where A* would pay 12." On screen the greedy route is orange, A*'s route is drawn in yellow beside it, and the strip says "best-first heads straight for the goal, cost X. A* (yellow) would go round for Y."

5. Breadth-first search (BFS)

`search.py breadth_first, within` course

What. Spreads out one step at a time (a FIFO queue), so it finds the route with the fewest steps.
Where. (a) *Hover* units (Cloud Drone, Electron, Quadcopter, Survey Drone) use it both to choose a target and to route to it: road, grass and mud cost them the same (the river still blocks them; they cross on bridges), so every step costs 1 and fewest steps is also cheapest. (b) `within(tile, 1)`, a breadth-first search with a step limit, gives the splash area of Logic Bomber, Oscillator, Steel Rain Cannon and Seismic Crew.
Why. When every step costs the same, BFS is optimal and simplest; a step-limited BFS is exactly "every tile within n steps". A hover unit's search shows as pink tiles; a splash makes its centre tile glow pink.

6. Fuzzy inference (Mamdani)

`fuzzy.py TARGET, priority` Lab 05

What. Written by hand in the lab's four steps: fuzzify, evaluate rules (OR = max, AND = min), aggregate (cap each output term at its rule's strength, then take the max), and defuzzify by the centroid over the samples $5, 15, \ldots, 95$. Inputs: *distance* in tiles (near/medium/far) and *health* in percent (low/medium/full). Output: *priority* 0–100.

```
IF distance is near   OR health is low    THEN priority is high
IF distance is medium OR health is medium THEN priority is medium
IF distance is far    OR health is full   THEN priority is low
```

Where. Only when a unit has *two or more* targets in range (enemy units or the enemy tower): it hits the one with the highest priority; with one target in range it simply hits it.
Why. "Close and badly hurt" is a vague idea, and fuzzy rules turn it into a smooth score that can be explained rule by rule. The on-screen panel shows each rule's strength.
Example. An enemy on the same tile at full health scores 50.0; one 1 tile away at full health scores 41.9; one on the same tile at 50% health scores 69.4, so the wounded enemy is hit.

7. Alpha-beta search (minimax with pruning)

`alphabeta.py, game.py` Lab 04 + theory

What. The Lab 04 minimax program (`actions`, `result`, `terminal`, `utility`, `max_value`, `min_value`) plus α and β. A move is (unused card, lane); the left side is MAX and the right side is MIN; utility = left tower − right tower. A branch that can no longer change the choice is skipped, and the answer is exactly plain minimax's. It *searches to the end of the leg*, with no depth limit and no evaluation function.
Where. The *Alpha-beta* brain; the *Hint* button; and the review of every move ("turning point" = the card that lost the most against perfect play).
Why. The game is deterministic, two-player, zero-sum, fully visible and turn-based, which is exactly what minimax assumes. A leg has at most $8 \cdot 8 \cdot 6 \cdot 6 \cdot 4 \cdot 4 \cdot 2 \cdot 2 = 147{,}456$ endings, so searching to the end is possible, and pruning makes it fast.
Example. First card of CSE against ME (seed 4): 13,821 positions looked at and 5,874 moves skipped, where plain minimax would need up to 270,025; about 0.3 s.

8. Monte Carlo Tree Search (UCB1)

`mcts.py` Lab 08

What. The four steps from the slides, repeated: *selection* (walk down by the highest UCB1 $= \frac{t_i}{n_i} + C\sqrt{\frac{\ln N}{n_i}}$, $C = 2$; unvisited children first), *expansion* (the root at the start, then any visited unfinished leaf, gets a child for every action, and the first new child is simulated), *simulation* (random moves, card and lane, for both sides until the leg ends) and *backpropagation* (add the result up the path).
Our choices (the slides leave them open): 1,500 repeats (Easy level: 100); a finished leg is worth 1 for a win, 0.5 for a draw and 0 for a loss to the side searching; the move played is the child with the best average t/n; a finished leg reached again is simply re-scored.
Where. The *MCTS* brain and the *Easy* brain.
Why. It is the second brain taught in the course and the opposite of alpha-beta: it samples instead of proving. Nothing models the opponent (the lab taught none), so it gambles; alpha-beta beats it in about 85% of matches (`balance_check.py`: 88% as the left side, 84% as the right, 80 matches each), which is the point of the show.

Not used: no evaluation function, no depth limit, no chance in the rules (the only randomness: the match seed shuffles the order the deck CSP reads the cards, and MCTS's rollouts), no hidden information, no AI libraries. Godot only draws the board: every rule and every algorithm runs in Python, and Godot asks `ai/bridge.py` once per decision.